

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»

Отчет по лабораторной работе №3-4
«Функциональные возможности языка Python.»

Выполнил:
Студент группы ИУ5-31Б
Шанурина Екатерина

Проверил:
Гапанюк Ю.Е.

2025 г.

Задание

Задание лабораторной работы состоит из решения нескольких задач. Файлы, содержащие решения отдельных задач, должны располагаться в пакете `lab_python_fr`. Решение каждой задачи должно располагаться в отдельном файле.

При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

Задача 1 (файл `field.py`)

Необходимо реализовать генератор `field`. Генератор `field` последовательно выдает значения ключей словаря. Пример:

- В качестве первого аргумента генератор принимает список словарей, дальше через `*args` генератор принимает неограниченное количество аргументов.
- Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно `None`, то элемент пропускается.
- Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно `None`, то оно пропускается. Если все поля содержат значения `None`, то пропускается элемент целиком.

Задача 2 (файл `gen_random.py`)

Необходимо реализовать генератор `gen_random`(количество, минимум, максимум), который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона. Пример:

`gen_random(5, 1, 3)` должен выдать 5 случайных чисел в диапазоне от 1 до 3, например 2, 2, 3, 2, 1

Задача 3 (файл `unique.py`)

- Необходимо реализовать итератор `Unique`(данные), который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.
- Конструктор итератора также принимает на вход именованный `bool`-параметр `ignore_case`, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен `False`.
- При реализации необходимо использовать конструкцию `**kwargs`.
- Итератор должен поддерживать работу как со списками, так и с генераторами.
- Итератор не должен модифицировать возвращаемые значения.

Задача 4 (файл sort.py)

Дан массив 1, содержащий положительные и отрицательные числа.

Необходимо **одной строкой кода** вывести на экран массив 2, который содержит значения массива 1, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции `sorted`.

Пример:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
```

```
Вывод: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
```

Необходимо решить задачу двумя способами:

1. С использованием `lambda`-функции.
2. Без использования `lambda`-функции.

Задача 5 (файл print_result.py)

Необходимо реализовать декоратор `print_result`, который выводит на экран результат выполнения функции.

- Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.
- Если функция вернула список (`list`), то значения элементов списка должны выводиться в столбик.
- Если функция вернула словарь (`dict`), то ключи и значения должны выводиться в столбик через знак равенства.

Задача 6 (файл cm_timer.py)

Необходимо написать контекстные менеджеры `cm_timer_1` и `cm_timer_2`, которые считают время работы блока кода и выводят его на экран. Пример:

```
with cm_timer_1():
```

```
    sleep(5.5)
```

После завершения блока кода в консоль должно вывестись `time:`

5.5 (реальное время может несколько отличаться).

`cm_timer_1` и `cm_timer_2` реализуют одинаковую функциональность, но должны быть реализованы двумя различными способами (на основе класса и с использованием библиотеки `contextlib`).

Задача 7 (файл process_data.py)

- В предыдущих задачах были написаны все требуемые инструменты для работы с данными. Применим их на реальном примере.
- В файле `data_light.json` содержится фрагмент списка вакансий.
- Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.
- Необходимо реализовать 4 функции - `f1`, `f2`, `f3`, `f4`. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора `@print_result` печатается результат, а контекстный менеджер `cm_timer_1` выводит время работы цепочки функций.

- Предполагается, что функции f1, f2, f3 будут реализованы в одну строку. В реализации функции f4 может быть до 3 строк.
- Функция f1 должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр. Используйте наработки из предыдущих задач.
- Функция f2 должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова “программист”. Для фильтрации используйте функцию filter.
- Функция f3 должна модифицировать каждый элемент массива, добавив строку “с опытом Python” (все программисты должны быть знакомы с Python). Пример: Программист С# с опытом Python. Для модификации используйте функцию map.
- Функция f4 должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Пример: Программист С# с опытом Python, зарплата 137287 руб. Используйте zip для обработки пары специальность — зарплата.

Листинг кода

field.py

```
def field(items, *args):
    assert len(args) > 0

    if len(args) == 1:
        key = args[0]
        for item in items:
            value = item.get(key)
            if value is not None:
                yield value
    else:
        for item in items:
            result = {}
            for key in args:
                value = item.get(key)
                if value is not None:
                    result[key] = value

            if result:
                yield result

print("Задание 1")
goods = [
    {'title': 'Ковер', 'price': 2000, 'color': 'green'},
    {'title': 'Диван для отдыха', 'color': 'black'}
]
```

```

print("Тест 1 - один аргумент 'title':")
for value in field(goods, 'title'):
    print(value)

print("\nТест 2 - один аргумент 'price':")
for value in field(goods, 'price'):
    print(value)

print("\nТест 3 - два аргумента 'title', 'price':")
for value in field(goods, 'title', 'price'):
    print(value)

```

gen_random.py

```

import random

def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)

print("\nЗадание 2")
print("Генерируем 5 случайных чисел от 1 до 3:")
for num in gen_random(5, 1, 3):
    print(num, end=' ')
print()

print("\nГенерируем 10 случайных чисел от 10 до 20:")
for num in gen_random(10, 10, 20):
    print(num, end=' ')
print()

```

unique.py

```

from gen_random import gen_random

# Итератор для удаления дубликатов
class Unique(object):
    def __init__(self, items, **kwargs):
        self.items = iter(items)
        self.seen = set()
        self.ignore_case = kwargs.get('ignore_case', False)

    def __next__(self):
        while True:
            item = next(self.items)

            # Для сравнения используем ключ
            if self.ignore_case and isinstance(item, str):
                key = item.lower()

```

```

        else:
            key = item

        # Если элемент еще не встречался, добавляем его и возвращаем
        if key not in self.seen:
            self.seen.add(key)
            return item

    def __iter__(self):
        return self

print("\nЗадание 3")
print("Тест 1 - список с дубликатами:")
data = [1, 1, 1, 1, 1, 2, 2, 2, 2]
for item in Unique(data):
    print(item, end=' ')
print()

print("\nТест 2 - генератор случайных чисел:")
data = gen_random(10, 1, 3)
for item in Unique(data):
    print(item, end=' ')
print()

print("\nТест 3 - строки, ignore_case=False:")
data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
for item in Unique(data):
    print(item, end=' ')
print()

print("\nТест 4 - строки, ignore_case=True:")
data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
for item in Unique(data, ignore_case=True):
    print(item, end=' ')
print()

sort.py
data = [4, -30, 100, -100, 123, 1, 0, -1, -4]

print("\nЗадание 4")
def sort():
    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
    print("C lambda:", result_with_lambda)

sort()
result = sorted(data, key=abs, reverse=True)
print("Без lambda:", result)

```

print_result.py

```
def print_result(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(func.__name__)

        if isinstance(result, list):
            # Если результат - список, выводим элементы в столбик
            for item in result:
                print(item)
        elif isinstance(result, dict):
            # Если результат - словарь, выводим ключи и значения через =
            for key, value in result.items():
                print(f'{key} = {value}')
        else:
            # Иначе просто выводим результат
            print(result)

    return result

    return wrapper

@print_result
def test_1():
    return 1

@print_result
def test_2():
    return 'iu5'

@print_result
def test_3():
    return {'a': 1, 'b': 2}

@print_result
def test_4():
    return [1, 2]

print("\nЗадание 5")
print('!!!!!!!!')
test_1()
test_2()
test_3()
test_4()
```

cm_timer.py

```
import time
from contextlib import contextmanager
```

```

# Способ 1: На основе класса
class cm_timer_1:
    def __enter__(self):
        self.start_time = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed_time = time.time() - self.start_time
        print(f'time: {elapsed_time:.2f}')

# Способ 2: С использованием библиотеки contextlib
@contextmanager
def cm_timer_2():
    start_time = time.time()
    try:
        yield
    finally:
        elapsed_time = time.time() - start_time
        print(f'time: {elapsed_time:.2f}')

print("\nЗадание 6")
print("Тест cm_timer_1:")
with cm_timer_1():
    time.sleep(2.5)

print("\nТест cm_timer_2:")
with cm_timer_2():
    time.sleep(1.5)

```

app.py

```

import json
import sys
from field import field
from gen_random import gen_random
from unique import Unique
from sort import sort
from print_result import print_result
from cm_timer import cm_timer_1
import random

from typing import List, Dict, Any

with open("data.json", 'r', encoding='utf-8') as file:
    json_data = json.load(file)

def unique(data):
    unique_data = set()

```



```

    for dct in data:
        job = dct.get("job-name")
        if job and job not in unique_data:
            yield job
            unique_data.add(job)

def get_jobs(data: List[Dict]) -> List:
    return list(unique(data))

def unique_reverse_sort(data: List):
    res = sorted(data, key = lambda s: s.lower(), reverse=True)
    return res

def get_all_programmists(data: List):
    return list(filter(lambda s: "программист" in s.lower(), data))

def add_python_experience(data):
    return list(map(lambda job: job + " с опытом Python", data))

def add_salary(data):
    return list(zip(data, [random.randint(100_000, 200_000) for sal in
range(len(data))]))

def executor(data : Any, actions: List[callable]):
    res = data
    for a in actions:
        res = a(res)
    return res

print("\nЗадание 7")

if __name__ == "__main__":
    action = [
        get_jobs,
        unique_reverse_sort,
        get_all_programmists,
        add_python_experience,
        add_salary]
    print('\n'.join(f'{i[0]}, {i[1]}' for i in executor(json_data, action)))

```

Результат выполнения

```
ekaterina@Katysya:~/bmstu/labs_sem2$ python3 app.py
```

Задание 1

Тест 1 - один аргумент 'title':

Ковер

Диван для отдыха

Тест 2 - один аргумент 'price':

2000

Тест 3 - два аргумента 'title', 'price':

{'title': 'Ковер', 'price': 2000}

{'title': 'Диван для отдыха'}

Задание 2

Генерируем 5 случайных чисел от 1 до 3:

2 1 2 1 1

Генерируем 10 случайных чисел от 10 до 20:

17 11 20 15 17 18 18 17 19 13

Задание 3

Тест 1 - список с дубликатами:

1 2

Тест 2 - генератор случайных чисел:

3 1 2

Тест 3 - строки, ignore_case=False:

a A b B

Тест 4 - строки, ignore_case=True:

a b

Задание 4

С lambda: [123, 100, -100, -30, 4, -4, 1, -1, 0]

Без lambda: [123, 100, -100, -30, 4, -4, 1, -1, 0]

Задание 5

!!!!!!!

test_1

1

test_2

iu5

test_3

a = 1

b = 2

test_4

1

2

Задание 6

Тест cm_timer_1:

time: 2.50

Тест cm_timer_2:

time: 1.50

Задание 7

Старший программист с опытом Python, 100190
Системный программист (C, Linux) с опытом Python, 192065
Программист-разработчик информационных систем с опытом Python, 150157
Программист/ технический специалист с опытом Python, 149354
Программист/ Junior Developer с опытом Python, 119822
Программист C++/C#/Java с опытом Python, 129377
Программист C++ с опытом Python, 128569
Программист C# с опытом Python, 120438
Программист 1C с опытом Python, 187257
программист 1C с опытом Python, 168793
Программист / Senior Developer с опытом Python, 189738
Программист с опытом Python, 188148
программист с опытом Python, 157176
Помощник веб-программиста с опытом Python, 172669
педагог программист с опытом Python, 124524
Инженер-электронщик (программист АСУ ТП) с опытом Python, 102417
Инженер-программист САПОУ (java) с опытом Python, 169431
Инженер-программист ПЛИС с опытом Python, 159398
Инженер-программист ККТ с опытом Python, 199981
Инженер-программист 1 категории с опытом Python, 146918
Инженер-программист (Орехово-Зуевский филиал) с опытом Python, 105586
Инженер-программист (Клинский филиал) с опытом Python, 169883
инженер-программист с опытом Python, 198646
Инженер-программист с опытом Python, 199304
Инженер - программист АСУ ТП с опытом Python, 166666
инженер - программист с опытом Python, 167665
Ведущий программист с опытом Python, 110267
Ведущий инженер-программист с опытом Python, 174056
Веб-программист с опытом Python, 115060
веб-программист с опытом Python, 127615
Веб - программист (PHP, JS) / Web разработчик с опытом Python, 191618
Web-программист с опытом Python, 197698
1C программист с опытом Python, 118263